

수업 03 · 2026-10-15

Python 심화

반복하는 일을 함수로 묶고 CSV를 읽어 JSON으로 저장합니다. 값이 없거나 잘못된 경우도 함께 처리해 봅니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅니다.

Python 심화 · 함수와 파일 검사

2026-10-15 · Python 심화

반복하는 일을 함수로 묶고 CSV를 읽어 JSON으로 저장합니다. 값이 없거나 잘못된 경우도 함께 처리해 봅니다.

수업은 50분씩 6교시입니다. 중식은 11:50~13:00입니다. 원본은 그대로 두고 work 폴더의 파일에서 안내한 부분을 고칩니다. 계정이나 실제 장비 없이 연습용 자료를 사용합니다.

오늘의 연결

어제 읽은 조건과 반복을 함수 안에 묶고 파일을 처리하는 데 사용합니다.

먼저 구별할 화면

- 파일 탐색기는 저장한 파일의 위치를 찾습니다.
- 텍스트 편집기는 코드와 데이터를 읽고 고칩니다.
- 터미널은 명령을 입력하고 실제 실행 결과를 확인합니다.
- 웹 교안은 설명·원문·출력 예를 제공합니다. Python 실행기는 아닙니다.

참여 순서

1. 압축을 일반 폴더에 풀고 data·code·work의 위치를 유지합니다.
2. lesson의 개념을 읽고 activity-guide의 해당 활동 입력을 엽니다.
3. work의 답칸에 예상과 이유를 먼저 쓰고 지정한 값을 수정합니다.
4. 저장한 work 파일을 실행합니다. 실행할 수 없으면 예상한 결과를 강사의 출력이나 제공된 예와 비교합니다.
5. 확인 방식을 기록합니다. 개인 실행을 하지 않았으면 실행 성공으로 적지 않습니다.

도구를 준비하지 못했을 때

설치가 안 되면 제공된 코드와 출력으로 수업을 따라갑니다. 같은 입력값을 넣고 한 줄씩 처리한 뒤의 값을 종이에 적어 봅니다. 강사가 실행 환경을 준비하면 직접 실행해 확인합니다.

[교안](#) · [실습](#) · [용어](#)

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 답을 적어 보고 풀이와 비교하세요.

함수의 입력과 결과

질문: 함수 안에서 print만 했다면 호출한 곳이 그 숫자를 계산에 쓸 수 있나요?

풀이: 화면 표시와 return으로 돌려주는 결과를 구분합니다. 호출 결과를 다른 계산에 쓰려면 필요한 값을 돌려줘야 합니다.

파일 읽기

질문: CSV 칸에 28.0이 적혀 있으면 자동으로 float인가요?
풀이: 일반적인 CSV 읽기는 문자열 값을 줍니다. 공백·누락·변환 실패를 확인한 뒤 숫자로 바꿉니다.

미측정

질문: 빈 온도를 0으로 바꾸면 평균은 어떻게 되나요?
풀이: 측정하지 않은 기록이 실제 0도처럼 계산되어 평균을 낮춥니다. 측정 여부와 정상 숫자를 따로 기록합니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 사례와 풀이를 읽고 필수 활동을 해 봅니다. 결과를 확인했으면 추가 문제로 복습하세요. 모든 페이지를 수업 시간 안에 마칠 필요는 없습니다. 실행이 안 되면 제공된 출력과 비교하며 코드의 처리 순서를 적어 봅니다. 직접 실행했는지는 따로 표시합니다.

2026-10-15 · Python 심화

반복하는 일을 함수로 묶고 CSV를 읽어 JSON으로 저장합니다. 값이 없거나 잘못된 경우도 함께 처리해 봅니다.

**확정 운영은 50분×6교시·순수 수업 300분이며 중식은 11:50~13:00입니다. 계정이나 실제 장비 없이 연습용 자료를 사용합니다.

1교시 · 반복 업무와 함수

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A02 · def로 정의하고 이름으로 호출하기

개념. def는 함수를 정의하는 문장입니다. 이름 뒤 괄호와 콜론을 쓰고 들여쓴 줄을 함수 본문으로 묶습니다. 정의만 하면 본문은 아직 실행되지 않으며 이름을 괄호와 함께 불러야 합니다.

왜 필요한가. 위에서 아래로 읽는 원리는 유지하지만 함수 본문은 호출할 때 실행됩니다. 정의 위치와 실제 실행 시점을 나누어 보면 출력 순서가 이해됩니다.

함께 보는 예. show_notice를 정의한 뒤 준비를 출력합니다. 함수 호출 때 안내가 나오고 호출에서 돌아온 뒤 끝이 나옵니다. 함수 본문의 줄 번호가 위에 있다고 가장 먼저 출력되는 것은 아닙니다.

헷갈리는 경우. show_notice라고 이름만 적는 것과 show_notice()로 호출하는 것은 다릅니다. 괄호는 실제 작업을 요청하는 표기입니다.

전체 코드

```
def show_notice():
    print("점검을 시작합니다")
print("준비")
show_notice()
print("끝")
```

각 줄의 뜻

줄	코드	뜻
1	def show_notice():	안내하는 작업을 정의합니다.
2	print("점검을 시작합니다")	호출할 때 실행할 출력입니다.
3	print("준비")	바깥에서 준비를 출력합니다.
4	show_notice()	정의한 함수를 호출합니다.
5	print("끝")	돌아온 뒤 끝을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A02_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A02_example.py
```

원본 코드의 예상 출력:

```
준비
점검을 시작합니다
끝
```

작은 수정: `show_notice()` `print("끝")`를 `show_notice()` `show_notice()` `print("끝")`로 바꿉니다.

수정 후 예상 출력:

```
준비
점검을 시작합니다
점검을 시작합니다
끝
```

학생 적용

1. 정의와 호출을 표시합니다.
2. 실제 출력 순서를 씁니다.
3. 호출을 한 번 더 넣어 예상합니다.
4. 정의 횟수와 본문 실행 횟수를 설명합니다.

예상 결과: 정의 한 번·호출 두 번이면 안내 출력 두 번.

확인 질문: 정의만 하고 호출하지 않으면 본문 출력은 나오나요?

막혔을 때: 본문 출력이 예상과 다르면 `print`가 함수 안에 들여써져 있는지 확인합니다.

[활동 자료와 작성칸](#)에서 A02를 엽니다.

A02 · 직접 해보기

A02 · def로 정의하고 이름으로 호출하기

목적: 위에서 아래로 읽는 원리는 유지하지만 함수 본문은 호출할 때 실행됩니다. 정의 위치와 실제 실행 시점을 나누어 보면 출력 순서가 이해됩니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A02_input.md](#)
- [A02_example.py](#)
- [A02_notes.md](#)
- [A02_example.py](#)

단계

1. 정의와 호출을 표시합니다.
2. 실제 출력 순서를 씁니다.
3. 호출을 한 번 더 넣어 예상합니다.
4. 정의 횟수와 본문 실행 횟수를 설명합니다.

예상 결과: 정의 한 번·호출 두 번이면 안내 출력 두 번.

확인 방법: 정의만 하고 호출하지 않으면 본문 출력은 나오나요?

오류 해결: 본문 출력이 예상과 다르면 print가 함수 안에 들여써져 있는지 확인합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A03 · 매개변수와 실제 인자

개념. 매개변수는 받을 값의 이름이고 인자는 호출할 때 실제로 주는 값입니다. 양식의 '장소' 칸과 거기에 넣는 '입구'라는 내용의 관계에 비유할 수 있습니다.

왜 필요한가. 고정 값을 함수 안에서 매번 바꾸는 대신 호출값을 달리해 같은 작업을 사용합니다. 함수 안 이름이 어디서 값을 받는지 알아야 전달한 값을 추적할 수 있습니다.

함께 보는 예. sensor_id와 value는 받을 이름입니다. 첫 호출에서 S01·22.5가 연결되고 다음 호출에서는 S02·25.0이 연결됩니다. 각 호출은 서로 다른 값으로 같은 본문을 사용합니다.

헷갈리는 경우. 인자의 순서를 바꾸어도 컴퓨터가 업무 의미를 추측해 바로잡지 않습니다. 센서명 위치에 숫자를 주면 잘못된 위치의 값이 출력될 수 있습니다.

전체 코드

```
def show_temperature(sensor_id, value):
    print(sensor_id, value)

show_temperature("S01", 22.5)

show_temperature("S02", 25.0)
```

각 줄의 뜻

줄	코드	뜻
1	<code>def show_temperature(sensor_id, value):</code>	센서명과 값을 받는 함수를 정의합니다.
2	<code>print(sensor_id, value)</code>	이번 호출에서 받은 값을 출력합니다.
3	<code>show_temperature("S01", 22.5)</code>	첫 센서와 온도를 전달합니다.
4	<code>show_temperature("S02", 25.0)</code>	둘째 센서와 온도를 전달합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A03_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A03_example.py
```

원본 코드의 예상 출력:

```
S01 22.5
S02 25.0
```

작은 수정: `show_temperature("S02", 25.0)`를 `show_temperature("S03", 0.0)`로 바꿉니다.

수정 후 예상 출력:

```
S01 22.5
S03 0.0
```

학생 적용

1. 전달값과 받을 이름을 짝짓습니다.
2. 둘째 입력을 S03-0.0으로 바꿉니다.
3. 값 하나가 부족한 호출과 정상 호출을 비교합니다.
4. 함수 내부를 고치지 않고 결과가 달라지는 이유를 씁니다.

예상 결과: 둘째 출력 S03 0.0. 인자 부족은 `TypeError`.

확인 질문: `value`는 모든 호출에서 같은 값을 뜻하나요?

막혔을 때: 받을 값 개수와 전달한 값 개수·순서를 비교합니다.

[활동 자료와 작성칸](#)에서 A03를 엽니다.

A03 · 직접 해보기

A03 · 매개변수와 실제 인자

목적: 고정 값을 함수 안에서 매번 바꾸는 대신 호출값을 달리해 같은 작업을 사용합니다. 함수 안 이름이 어디서 값을 받는지 알아야 전달한 값을 추적할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A03_input.md](#)
- [A03_example.py](#)
- [A03_notes.md](#)
- [A03_example.py](#)

단계

1. 전달값과 받을 이름을 짝꿍합니다.
2. 둘째 입력을 S03-0.0으로 바꿉니다.
3. 값 하나가 부족한 호출과 정상 호출을 비교합니다.
4. 함수 내부를 고치지 않고 결과가 달라지는 이유를 씁니다.

예상 결과: 둘째 출력 S03 0.0. 인자 부족은 TypeError.

확인 방법: value는 모든 호출에서 같은 값을 뜻하나요?

오류 해결: 받을 값 개수와 전달한 값 개수·순서를 비교합니다.

수정 전 예상: _____
 확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 같은 업무를 함수로 묶는 이유

[이 교시의 다른 예와 코드 해설 열기](#)

2교시 · 함수 입력과 반환

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A04 · print와 return의 차이

사례로 이해하기 · 화면 표시와 다음 계산의 반환값 구별하기

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

청구서 담당자는 수량 3과 단가 1200원으로 상품 금액을 계산한 뒤 배송비 2000원을 더합니다. 첫 계산이 화면에 3600을 보여 주었다고 다음 계산에서 그 값을 사용할 수 있는 것은 아닙니다. 화면 표시는 사람에게 보여 주는 행동이고 반환은 함수를 부른 곳에 결과 값을 보내는 행동입니다. 전자계산기에서 값을 읽는 것과 그 값을 다음 식의 입력으로 전달하는 것을 구별합니다. 필요한 최종 결과를 먼저 쓰면 print와 return의 역할이 선명해집니다.

이 함수는 단가와 수량을 정수로 받아 상품 금액을 정수로 돌려줍니다. 수업용 처리 규칙은 단가×수량이며 세금이나 할인은 넣지 않습니다. 호출에서 1200과 3을 주면 함수 안에서 3600을 만들고 return으로 보냅니다. 호출한 쪽은 받은 값에 2000을 더해 최종 5600원을 얻습니다. 손 추적표는 입력 1200·3 → 함수 계산 3600 → 반환 3600 → 배송비 포함 5600 순서입니다. 화면에 무엇이 보이는가와 변수에 무엇이 연결되는가를 다른 칸으로 적습니다.

print만 하는 함수는 3600을 화면에 보이게 해도 별도 return이 없으면 호출 결과가 None입니다. None에 2000을 더하면 숫자 계산이 되지 않습니다. 오류를 피하려고 None을 0으로 바꾸면 상품 금액이 사라져 잘못된 청구서가 됩니다. 결과를 계산하는 함수에 return을 두고 필요한 표시를 호출한 쪽에서 맡기는 구조를 생각합니다. 또한 return을 만나는 즉시 그 호출은 끝납니다. 그 아래 줄은 같은 호출에서 이어서 실행되지 않습니다. 조건이 여러 갈래인 함수라면 각 갈래가 무엇을 반환하는지 읽어야 합니다. 한 갈래에서만 반환하면 나머지 입력에서 뜻밖의 None이 생길 수 있습니다.

혼자 풀어 볼 문제

복습·도전: 배송 준비 함수는 포장비 500원을 화면에 표시하지만 반환문이 없습니다. 호출 결과를 fee에 연결한 뒤 총액 3000에 더하려 합니다. 화면값·fee 값·다음 계산을 분리해 설명하고 올바른 역할 배치를 문장으로 설계하세요.

정답과 오답의 이유

화면에는 500이 보이지만 fee는 None이며 다음 덧셈은 TypeError가 날 수 있습니다. 함수가 500을 반환하고 호출자가 총액에 더한 뒤 표시하도록 설계합니다. 화면에 500이 보이므로 fee도 500이라는 답은 표시와 반환을 혼동한 것입니다.

다시 확인할 질문 세 가지

1. **print의 결과가 자동 저장되나요?** 표시와 반환은 별도이며 호출 결과를 보려면 return을 확인합니다.
2. **return 뒤 안내가 실행되나요?** 같은 호출은 return에서 끝나므로 아래 줄까지 가지 않습니다.
3. **None을 0으로 고쳐도 되나요?** 업무 결과가 누락된 원인을 고쳐야 하며 임의 0은 계산 의미를 바꿉니다.

개념. print는 값을 화면에 보여 주고 return은 함수를 부른 위치로 값을 돌려줍니다. 화면에 결과가 보였다는 사실과 다음 계산에 사용할 결과를 받았다는 사실은 다릅니다.

왜 필요한가. 결과를 JSON에 저장하거나 집계하려면 함수가 값을 돌려줘야 합니다. 화면 글자를 다른 코드가 저절로 변수에 담지는 않습니다.

함께 보는 예. `add_offset`은 받은 두 수를 더해 반환합니다. `result`가 23.5를 받고 다음 계산에서 사용합니다. 더하는 값은 수업 계산 예이며 실제 장비 교정값으로 사용하지 않습니다.

헛갈리는 경우. `return` 없이 `print`만 하는 함수의 반환값은 `None`입니다. 그 값을 숫자처럼 더하려고 하면 오류가 날 수 있습니다. 화면만 보지 말고 `return`의 역할을 확인합니다.

전체 코드

```
def add_offset(value, offset):  
    return value + offset  
  
result = add_offset(22.5, 1.0)  
print(result)  
print(result + 1)
```

각 줄의 뜻

줄	코드	뜻
1	<code>def add_offset(value, offset):</code>	두 값을 받는 작업을 정의합니다.
2	<code>return value + offset</code>	합을 호출한 곳으로 돌려줍니다.
3	<code>result = add_offset(22.5, 1.0)</code>	돌아온 결과를 <code>result</code> 에 연결합니다.
4	<code>print(result)</code>	받은 결과를 출력합니다.
5	<code>print(result + 1)</code>	결과를 다음 계산에 사용합니다.

실행과 예상 출력

현재 위치가 압축을 푼 `course` 폴더일 때 실행합니다.

```
python -X utf8 code/A04_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A04_example.py
```

원본 코드의 예상 출력:

```
23.5  
24.5
```

작은 수정: `22.5, 1.0`를 `22.5, 2.0`로 바꿉니다.

수정 후 예상 출력:

```
24.5
25.5
```

학생 적용

1. 출력과 반환의 경로를 구별합니다.
2. result가 값을 받는 위치를 찾습니다.
3. offset을 2.0으로 바꿔 예상합니다.
4. 두 출력의 관계를 확인합니다.

예상 결과: 24.5를 반환하고 다음 계산은 25.5.

확인 질문: 화면에 보인 값은 언제나 같은 함수의 반환값인가요?

막혔을 때: None이 보이면 return이 있는지, 특정 경로에서 반환 없이 끝나는지 확인합니다.

[활동 자료와 작성칸](#)에서 A04를 엽니다.

A04 · 직접 해보기

A04 · print와 return의 차이

목적: 결과를 JSON에 저장하거나 집계하려면 함수가 값을 돌려줘야 합니다. 화면 글자를 다른 코드가 저절로 변수에 담지는 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A04_input.md](#)
- [A04_example.py](#)
- [A04_notes.md](#)
- [A04_example.py](#)

단계

1. 출력과 반환의 경로를 구별합니다.
2. result가 값을 받는 위치를 찾습니다.
3. offset을 2.0으로 바꿔 예상합니다.
4. 두 출력의 관계를 확인합니다.

예상 결과: 24.5를 반환하고 다음 계산은 25.5.

확인 방법: 화면에 보인 값은 언제나 같은 함수의 반환값인가요?

오류 해결: None이 보이면 return이 있는지, 특정 경로에서 반환 없이 끝나는지 확인합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 조건을 담은 상태 함수 / 예상 사례표로 함수 확인하기

[이 교시의 다른 예와 코드 해설 열기](#)

3교시 · 파일을 읽고 결과 저장

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A08 · CSV를 이름표가 있는 행으로 읽기

개념. csv.DictReader는 헤더를 키로 삼아 각 데이터 행을 딕셔너리로 읽습니다. 기본 셀값은 문자열이므로 온도 계산 전에 의미에 맞는 해석이 필요합니다.

왜 필요한가. 열 번호를 외우기보다 temperature_c라는 이름으로 읽으면 처리할 항목이 드러납니다. 실제 헤더가 약속과 맞아야 같은 이름으로 읽을 수 있습니다.

함께 보는 예. 입력은 S01의 22.5, S02의 25.0, S03의 빈 온도, S04의 0.0 네 건입니다. 첫 온도의 화면 모양은 숫자 같지만 type은 str입니다.

헷갈리는 경우. 헤더를 temperature로 바꾸면 temperature_c라는 키가 저절로 생기지 않습니다. DictReader가 업무상 비슷한 단어를 연결하지 않습니다.

전체 코드

```
import csv

from pathlib import Path

source = Path(__file__).resolve().parent.parent / "data" / "readings.csv"

with source.open(encoding="utf-8", newline="") as file:
    rows = list(csv.DictReader(file))

print(len(rows))

print(rows[0]["sensor_id"])

print(rows[0]["temperature_c"], type(rows[0]["temperature_c"]).__name__)
```

각 줄의 뜻

줄	코드	뜻
1	<code>import csv</code>	CSV 표준 도구를 가져옵니다.
2	<code>from pathlib import Path</code>	파일 주소 도구를 가져옵니다.
3	<code>source = Path(__file__).resolve().parent.parent / "data" / "readings.csv"</code>	기초 CSV 주소를 만듭니다.
4	<code>with source.open(encoding="utf-8", newline="") as file:</code>	인코딩과 CSV 줄바꿈 설정으로 엽니다.
5	<code>rows = list(csv.DictReader(file))</code>	각 행을 헤더 이름의 딕셔너리로 읽습니다.
6	<code>print(len(rows))</code>	데이터 건수를 출력합니다.
7	<code>print(rows[0]["sensor_id"])</code>	첫 기록의 센서명을 읽습니다.
8	<code>print(rows[0]["temperature_c"], type(rows[0]["temperature_c"]).__name__)</code>	첫 온도와 실제 자료형을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A08_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A08_example.py
```

원본 코드의 예상 출력:

```
4
S01
22.5 str
```

학생 적용

1. 헤더와 키의 대응을 표시합니다.
2. 헤더를 제외한 네 건을 셉니다.
3. 온도의 값 모양과 자료형을 나눠 적습니다.
4. 빈 온도가 어떤 문자열인지 예상합니다.

예상 결과: 4 / S01 /22.5 str. 헤더는 데이터가 아닙니다.

확인 질문: 빈 셀은 자동으로 None이 되나요?

막혔을 때: KeyError는 실제 헤더와 키를 비교합니다. 공백·인코딩도 실제 원문을 확인한 뒤 판단합니다.

[활동 자료와 작성칸](#)에서 A08를 엽니다.

A08 · 직접 해보기

A08 · CSV를 이름표가 있는 행으로 읽기

추가 원문·작업 파일:

- [A08_input.md](#)
- [A08_example.py](#)
- [readings.csv](#)
- [A08_notes.md](#)
- [A08_example.py](#)

목적: 열 번호를 외우기보다 temperature_c라는 이름으로 읽으면 처리할 항목이 드러납니다. 실제 헤더가 약속과 맞아야 같은 이름으로 읽을 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A08_input.md](#)
- [A08_example.py](#)
- [A08_notes.md](#)
- [A08_example.py](#)

단계

1. 헤더와 키의 대응을 표시합니다.
2. 헤더를 제외한 네 건을 셉니다.
3. 온도의 값 모양과 자료형을 나눠 적습니다.
4. 빈 온도가 어떤 문자열일지 예상합니다.

예상 결과: 4 / S01 /22.5 str. 헤더는 데이터가 아닙니다.

확인 방법: 빈 셀은 자동으로 None이 되나요?

오류 해결: KeyError는 실제 헤더와 키를 비교합니다. 공백·인코딩도 실제 원문을 확인한 뒤 판단합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A09 · JSON 결과를 새 파일로 저장하기

개념. json.dumps는 프로그램 값을 JSON 텍스트로 표현합니다. 그 텍스트를 경로에 써야 결과 파일이 생깁니다. 입력 읽기와 결과 저장은 별도 작업입니다.

왜 필요한가. 화면 출력만 있으면 다음 사람이 결과를 재사용하기 어렵습니다. 입력 원자료를 덮어쓰지 않도록 work의 결과 경로를 명확히 정합니다.

함께 보는 예. 건수 4와 미측정 1을 JSON으로 저장합니다. ensure_ascii=False는 한글을 읽기 쉬운 모양으로 남기고 indent=2는 들여쓰기를 넣습니다. 저장한 파일을 다시 읽어 값이 같은지 봅니다.

헛갈리는 경우. .json이라는 이름만으로 내용이 JSON이 되지는 않습니다. 딕셔너리 화면 문자열을 그대로 쓰기보다 JSON 변환 도구를 사용합니다.

전체 코드

```
import json

from pathlib import Path

base = Path(__file__).resolve().parent.parent

summary = {"건수": 4, "미측정": 1}

text = json.dumps(summary, ensure_ascii=False, indent=2)

target = base / "work" / "output_summary.json"

target.write_text(text, encoding="utf-8")

loaded = json.loads(target.read_text(encoding="utf-8"))

print(loaded["건수"], loaded["미측정"])
```

각 줄의 뜻

줄	코드	뜻
1	<code>import json</code>	JSON 도구를 가져옵니다.
2	<code>from pathlib import Path</code>	주소 도구를 가져옵니다.
3	<code>base = Path(__file__).resolve().parent.parent</code>	course를 찾습니다.
4	<code>summary = {"건수": 4, "미측정": 1}</code>	결과 객체를 만듭니다.
5	<code>text = json.dumps(summary, ensure_ascii=False, indent=2)</code>	JSON 텍스트로 표현합니다.
6	<code>target = base / "work" / "output_summary.json"</code>	work의 결과 주소를 정합니다.
7	<code>target.write_text(text, encoding="utf-8")</code>	UTF-8로 저장합니다.
8	<code>loaded = json.loads(target.read_text(encoding="utf-8"))</code>	저장한 파일을 다시 읽어 해석합니다.
9	<code>print(loaded["건수"], loaded["미측정"])</code>	다시 읽은 두 값을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A09_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A09_example.py
```

원본 코드의 예상 출력:

```
4 1
```

작은 수정: "건수": 4를 "건수": 5로 바꿉니다.

수정 후 예상 출력:

```
5 1
```

학생 적용

1. data와 work의 역할을 구별합니다.
2. 실행 후 output_summary.json을 엽니다.
3. 건수만 5로 바꾸고 다시 실행합니다.

4. 파일 저장 줄과 화면 출력 줄을 표시합니다.

예상 결과: 처음 4 1, 수정 후 5 1. 저장 파일의 숫자도 바뀝니다.

확인 질문: print와 write_text는 같은 행동인가요?

막혔을 때: 쓰기 권한이 없으면 강사와 저장 위치를 확인합니다. 원자료를 출력 주소로 바꾸지 않습니다.

[활동 자료와 작성칸](#)에서 A09를 엽니다.

A09 · 직접 해보기

A09 · JSON 결과를 새 파일로 저장하기

추가 원문·작업 파일:

- [A09_input.md](#)
- [A09_example.py](#)
- [A09_notes.md](#)
- [A09_example.py](#)
- [output_summary.json](#)

목적: 화면 출력만 있으면 다음 사람이 결과를 재사용하기 어렵습니다. 입력 원자료를 덮어쓰지 않도록 work의 결과 경로를 명확히 정합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A09_input.md](#)
- [A09_example.py](#)
- [A09_notes.md](#)
- [A09_example.py](#)

단계

1. data와 work의 역할을 구별합니다.
2. 실행 후 output_summary.json을 엽니다.
3. 건수만 5로 바꾸고 다시 실행합니다.
4. 파일 저장 줄과 화면 출력 줄을 표시합니다.

예상 결과: 처음 4 1, 수정 후 5 1. 저장 파일의 숫자도 바뀝니다.

확인 방법: print와 write_text는 같은 행동인가요?

오류 해결: 쓰기 권한이 없으면 강사와 저장 위치를 확인합니다. 원자료를 출력 주소로 바꾸지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 파일 경로와 텍스트 읽기

이 교시의 다른 예와 코드 해설 열기

4교시 · 오류를 단서로 읽기

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A10 · 미측정과 잘못된 숫자의 구별

사례로 이해하기 · 미측정·잘못된 글자·실제 0의 세 갈래

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

수업용 측정 파일에 빈 문자열, "0", "추움" 세 원문이 있습니다. 검사를 통과시키려고 모두 0으로 바꾸면 세 기록이 실제 0°C로 합쳐집니다. 빈 문자열은 이번 데이터 사전에서 미측정이고, "0"은 측정한 숫자 표현이며, "추움"은 숫자 칸에 온 다른 종류의 내용입니다. 먼저 원문을 보존하고 어떤 갈래에 속하는지 정합니다. 오류 처리는 숨기는 행동이 아니라 분류와 후속 확인을 기록하는 행동입니다.

해석 절차의 첫 질문은 원문이 빈 문자열과 정확히 같은지입니다. 같으면 None으로 반환해 미측정을 보존합니다. 아니면 숫자로 변환합니다. "0"은 0.0이 되지만 "추움"은 ValueError가 납니다. 이 예에서는 ValueError를 잡아 장치번호와 원문을 이슈로 남깁니다. 처리할 수 없는 숫자 글자 오류를 잡는 것과 주소 오류까지 무조건 숨기는 것은 다릅니다. except ValueError는 해당 변환·규칙 오류만 말고 파일이 없다는 오류의 해결은 별도입니다.

완성 분류표는 R01/빈 문자열/미측정, R02/"0"/수치 0.0, R03/"추움"/이슈입니다. 수치 평균을 계산한다면 R02만 대상으로 하므로 평균은 0.0입니다. R01을 0으로 채워 평균에 넣으면 대상 수가 늘고 미측정 사실이 사라집니다. R03을 None으로 바꾸면 잘못 적힌 내용이 있었다는 근거가 사라집니다. 오류 원문을 보존한 후 담당자에게 어떤 입력을 의도했는지 묻습니다. 공백 한 칸 " "도 빈 문자열과 그대로는 다르며 이 수업의 규칙에는 자동 공백 정리가 없습니다. 새 정리 규칙이 필요하면 먼저 원문과 변경 이유를 기록하고 별도 약속으로 정해야 합니다.

혼자 풀어 볼 문제

복습·도전: 수량 칸 원문이 빈 문자열, "0", "여섯", " "입니다. 이 문제의 규칙은 정확한 빈 문자열만 미확인으로 보고, 숫자만 계산 대상으로 정합니다. 네 갈래의 분류와 담당자에게 남길 이슈 원문을 작성하세요.

정답과 오답의 이유

빈 문자열은 미확인, "0"은 수치 0, "여섯"과 공백 한 칸은 현재 규칙으로 숫자로 읽을 수 없는 입력입니다. 두 이슈 원문을 그대로 남깁니다. 공백을 자동 삭제해 미확인으로 합치는 답은 명시되지 않은 정리 규칙을 추가한 것입니다.

다시 확인할 질문 세 가지

1. **미측정도 오류인가요?** 규칙에서 허용한 빈값과 잘못된 입력을 구별합니다.
2. **예외를 잡으면 실제 값을 알게 되나요?** 아닙니다. 실패한 입력값을 그대로 남겨 무엇을 더 확인할지 알 수 있습니다.
3. **0을 평균에서 빼나요?** 실제 수치 0은 포함하고 값 없음만 제외합니다.

개념. 빈값 규칙과 숫자 변환은 다른 단계입니다. 이 자료의 빈 온도는 미측정이므로 None으로 보존하고 숫자가 아닌 글자는 잘못된 입력으로 알립니다.

왜 필요한가. 모든 변환 실패를 None으로 바꾸면 누락과 잘못된 기록을 구별하지 못합니다. 원문과 실패 이유를 남겨야 확인 요청을 할 수 있습니다.

함께 보는 예. 빈 문자열은 None, "0.0"은 숫자 0.0, "뜨거움"은 ValueError입니다. 먼저 빈 문자열인지 확인하고 나머지에 숫자 변환을 적용합니다.

헛갈리는 경우. 빈 문자열을 0으로 대신 채우면 미측정을 실제 0°C로 바꿉니다. 계산을 편하게 하려고 확인되지 않은 값을 보충하지 않습니다.

전체 코드

```
def parse_temperature(raw):
    if raw == "":
        return None
    return float(raw)
print(parse_temperature("22.5"))
print(parse_temperature(""))
print(parse_temperature("0.0"))
```

각 줄의 뜻

줄	코드	뜻
1	<code>def parse_temperature(raw):</code>	원문 온도를 해석하는 함수를 정의합니다.
2	<code>if raw == "":</code>	빈 문자열 여부를 확인합니다.
3	<code>return None</code>	계약에 따라 None을 반환합니다.
4	<code>return float(raw)</code>	나머지를 숫자로 변환합니다.
5	<code>print(parse_temperature("22.5"))</code>	일반 숫자를 확인합니다.
6	<code>print(parse_temperature(""))</code>	미측정을 확인합니다.
7	<code>print(parse_temperature("0.0"))</code>	실제 0을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A10_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A10_example.py
```

원본 코드의 예상 출력:

```
22.5
None
0.0
```

학생 적용

1. 원문·반환값·업무 의미를 표로 적습니다.
2. 뜨거움과 빈칸의 차이를 설명합니다.
3. 오류 원문과 종류를 기록합니다.
4. 실제값이 미확인일 때 남길 질문을 씁니다.

예상 결과: 22.5/None/0.0을 구별하며 글자는 ValueError.

확인 질문: 잘못된 숫자를 미측정으로 바꾸면 무엇을 잃나요?

막혔을 때: None과 문자열 "None"은 다릅니다. 후자는 숫자로 읽으면 실패합니다.

[활동 자료와 작성칸](#)에서 A10를 엽니다.

A10 · 직접 해보기

A10 · 미측정과 잘못된 숫자의 구별

목적: 모든 변환 실패를 None으로 바꾸면 누락과 잘못된 기록을 구별하지 못합니다. 원문과 실패 이유를 남겨야 확인 요청을 할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A10_input.md](#)

- [A10_example.py](#)

- [A10_notes.md](#)

- [A10_example.py](#)

단계

1. 원문·반환값·업무 의미를 표로 적습니다.
2. 뜨거움과 빈칸의 차이를 설명합니다.
3. 오류 원문과 종류를 기록합니다.

4. 실제값이 미확인일 때 남길 질문을 씁니다.

예상 결과: 22.5/None/0.0을 구별하며 글자는 ValueError.

확인 방법: 잘못된 숫자를 미측정으로 바꾸면 무엇을 잃나요?

오류 해결: None과 문자열 "None"은 다릅니다. 후자는 숫자로 읽으면 실패합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 오류의 위치·종류·원인 / try와 except로 예상 오류 기록

[이 교시의 다른 예와 코드 해설 열기](#)

5교시 · 검사 규칙과 재확인

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A13 · 범위 규칙을 함수에서 확인하기

사례로 이해하기 · 허용 범위의 바깥과 경계를 검사하기

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

접수 시스템은 물품 수량을 0개부터 20개까지 받습니다. 21은 숫자로 바꿀 수 있지만 정해 둔 수량 범위에는 맞지 않습니다. 숫자로 바꿀 수 있는지와 허용 범위 안인지는 따로 검사합니다. 먼저 "스물"은 숫자 표현이 아니라 변환에서 멈추고, 21은 숫자로 읽힌 뒤 범위에서 멈춥니다. 0은 실제 수량이 없다는 뜻이며 이번 규칙에서는 허용합니다. 같은 ValueError라는 결과가 나올 수 있어도 어떤 규칙과 원문 때문에 실패했는지 기록합니다.

$0 \leq \text{수량} \leq 20$ 을 수직선으로 그리면 두 끝점이 포함됩니다. 바깥은 '0보다 작음 또는 20보다 큼'입니다. or는 둘 중 하나라도 참이면 참이고 and는 둘 다 참이어야 참입니다. 완성 검수표는 -1 거부, 0 허용, 10 허용, 20 허용, 21 거부입니다. 경계 0과 20을 보지 않으면 <와 <=를 바꾼 실수를 놓칠 수 있습니다. '0보다 작고 동시에 20보다 큼'이라는 and 조건은 한 숫자에 성립하지 않아 범위 밖을 놓칩니다.

20이 재고 한도인지 주문 수량 한도인지도 규칙에 적어야 합니다. 같은 숫자 20을 모든 업무의 기준으로 재사용하지 않습니다. 21을 자동으로 20으로 잘라 저장하면 검사 대신 원자료 변경을 수행한 것입니다. 담당자의 확인 없이 실제 수량을 고치지 않고 거부 사유와 원문을 남깁니다. 수정 검증은 실패 입력 한 건만 다시 보는 것이 아닙니다. 기준 비교를 바꿨다면 정상·양쪽 경계·양쪽 바깥의 다섯 사례를 다시 봅니다. 통과한 사례 수와 검사하지 않은 종류를 구별해서 보고합니다. 이 예제는 정수 개수를 받습니다. 1.5개 같은 값을 허용할지는 따로 정해야 합니다. 범위 통과만으로 정수 조건이나 실물 재고의 정확성이 증명되지는 않습니다.

혼자 풀어 볼 문제

복습·도전: 수업용 배터리 기록의 허용 충전율은 0~100%입니다. -0.1, 0, 50, 100, 100.1의 기대 결과와 바깥을 찾는 논리 문장을 쓰세요. 100.1을 100으로 자동 변경해도 되는지 근거를 설명하세요.

정답과 오답의 이유

양 끝 -0.1·100.1은 거부, 0·50·100은 허용입니다. '0 미만 또는 100 초과'가 바깥입니다. 자동 변경은 원문을 바꾸므로 현재 검사 규칙에서는 허용하지 않습니다. and로 쓰면 두 바깥 조건을 동시에 만족하는 값이 없어 오류를 놓칩니다.

다시 확인할 질문 세 가지

- 1. 변환 성공이 허용 성공인가요? 숫자 표현과 업무 범위·정수 조건은 별도 검사입니다.
- 2. 범위 밖에 왜 or를 쓰나요? 아래쪽이나 위쪽 중 하나만 벗어나도 거부해야 하기 때문입니다.
- 3. 통과가 실제 정확성을 뜻하나요? 입력 규칙에 맞았다는 뜻이며 실제 측정·기록 정확성은 따로 확인합니다.

개념. 숫자로 변환할 수 있다는 것과 허용 범위 안에 있다는 것은 다른 검사입니다. 값이 0~50인지 확인하고 벗어나면 이유를 예외로 알립니다.

왜 필요한가. 51.0은 숫자이므로 float는 성공하지만 실습 입력 규칙에는 맞지 않습니다. 변환과 규칙 검사를 분리하여 읽어야 합니다.

함께 보는 예. 23.5는 반환하고 51.0은 범위 확인 필요를 알립니다. 0과 50은 허용 경계입니다. 이 범위는 실습의 입력 약속이며 실제 장비 안전 기준이 아닙니다.

헛갈리는 경우. 51.0을 자동으로 50으로 자르면 원자료를 바꿉니다. 실제값을 모르면 원문을 보존하고 확인 요청을 남깁니다.

전체 코드

```
def checked_temperature(raw):  
    value = float(raw)  
    if value < 0 or value > 50:  
        raise ValueError("범위 확인 필요")  
    return value  
  
print(checked_temperature("23.5"))
```

각 줄의 뜻

줄	코드	뜻
1	def checked_temperature(raw):	수치 원문을 검사하는 함수를 정의합니다.
2	value = float(raw)	숫자로 해석합니다.
3	if value < 0 or value > 50:	아래 또는 위 경계를 벗어나는지 묻습니다.
4	raise ValueError("범위 확인 필요")	범위 문제를 예외로 알립니다.
5	return value	허용 숫자를 반환합니다.
6	print(checked_temperature("23.5"))	정상 입력으로 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A13_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A13_example.py
```

원본 코드의 예상 출력:

```
23.5
```

작은 수정: "23.5"를 "50"로 바꿉니다.

수정 후 예상 출력:

```
50.0
```

학생 적용

1. 변환과 범위 검사를 표시합니다.
2. 0·50·51.0의 기대 결과를 씁니다.
3. 경계와 오류 입력을 비교합니다.
4. 원문과 확인 이유를 남깁니다.

예상 결과: 0·50은 허용하고 51.0은 범위 오류입니다.

확인 질문: 범위 통과가 실제 센서 정확도를 증명하나요?

막혔을 때: or를 and로 바꿨다면 두 조건의 뜻을 읽습니다. 동시에 0보다 작고 50보다 큰 값은 찾을 수 없습니다.

[활동 자료와 작성칸](#)에서 A13를 엽니다.

A13 · 직접 해보기

A13 · 범위 규칙을 함수에서 확인하기

목적: 51.0은 숫자이므로 float는 성공하지만 실습 입력 규칙에는 맞지 않습니다. 변환과 규칙 검사를 분리하여 읽어야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A13_input.md](#)
- [A13_example.py](#)
- [A13_notes.md](#)
- [A13_example.py](#)

단계

1. 변환과 범위 검사를 표시합니다.
2. 0.50-51.0의 기대 결과를 씁니다.
3. 경계와 오류 입력을 비교합니다.
4. 원문과 확인 이유를 남깁니다.

예상 결과: 0.50은 허용하고 51.0은 범위 오류입니다.

확인 방법: 범위 통과가 실제 센서 정확도를 증명하나요?

오류 해결: or를 and로 바꿨다면 두 조건의 뜻을 읽습니다. 동시에 0보다 작고 50보다 큰 값은 찾을 수 없습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A14 · 한 값 수정 뒤 같은 사례 재확인

개념. 수정 검증은 확인된 근거로 한 부분을 바꾸고 실패 사례와 기존 성공 사례를 다시 확인하는 일입니다. 오류가 사라졌는지뿐 아니라 원래 의미가 유지되는지 봅니다.

왜 필요한가. 한 문제를 고치며 미측정을 0으로 바꾸거나 경계를 바꿀 수 있습니다. 원문·지시·수정 위치·재확인을 함께 남겨야 이유가 전달됩니다.

함께 보는 예. 지시서에서 S05의 ‘뜨거움’은 23.5로 확정되었고 S06의 51.0은 미확인입니다. 전자는 근거로 수정하고 후자는 원문 이슈로 남깁니다.

헛갈리는 경우. 문제가 두 건이라고 둘 다null로 바꾸면 잘못된 숫자와 미측정을 합칩니다. 확인된 수정과 미확인 보류를 구별합니다.

제공 입력

확정 지시: S05 10:05는23.5°C로 확인됨.

미확정: S06 10:05의51.0은 실제값미확인.

재확인: 정상22.5 / 빈칸 / 실제0.0 / 경계50.

학생 적용

1. 확정과 미확정 지시를 나눕니다.
2. work/correction_notes.md에 전후값과 근거를 씁니다.
3. work/bad_readings_work.csv에서 S05 한 값만 고칩니다.
4. 정상·미측정·0·범위의 재확인 표를 작성합니다.

예상 결과: S05만 23.5로 수정하고 S06원문과 확인 이유를 남깁니다.

확인 질문: 이슈 건수만 줄면 좋은 수정인가요?

막혔을 때: 확정되지 않은 실제값은 추측하지 않습니다. 어느 값이 왜 미확인인지 씁니다.

[활동 자료와 작성칸](#)에서 A14를 엽니다.

A14 · 직접 해보기

A14 · 한 값 수정 뒤 같은 사례 재확인

추가 원문·작업 파일:

- [A14_input.md](#)
- [bad_readings.csv](#)
- [A14_notes.md](#)
- [bad_readings_work.csv](#)
- [correction_notes.md](#)

목적: 한 문제를 고치며 미측정을 0으로 바꾸거나 경계를 바꿀 수 있습니다. 원문·지시·수정 위치·재확인을 함께 남겨야 이유가 전달됩니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A14_input.md](#)
- [A14_notes.md](#)

단계

1. 확정과 미확정 지시를 나눕니다.
2. work/correction_notes.md에 전후값과 근거를 씁니다.
3. work/bad_readings_work.csv에서 S05 한 값만 고칩니다.
4. 정상·미측정·0·범위의 재확인 표를 작성합니다.

예상 결과: S05만 23.5로 수정하고 S06원문과 확인 이유를 남깁니다.

확인 방법: 이슈 건수만 줄면 좋은 수정인가요?

오류 해결: 확정되지 않은 실제값은 추측하지 않습니다. 어느 값이 왜 미확인인지 씁니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 어떤 파일을 넣고 어떤 결과를 받을지 적어 봅시다

[이 교시의 다른 예와 코드 해설 열기](#)

6교시 · 로그 검사기와 사용 안내

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A16 · 로그 검사기 전체 흐름

사례로 이해하기 · 분류 건수와 평균 분모를 보존하기

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

로그 검사기를 넘겨받은 사람이 요약의 accepted를 실제 숫자 수라고 읽었습니다. 그러나 기존 검사기에서 accepted는 수치와 미측정을 합한 수락 건수입니다. 이름만 보고 뜻을 짐작하지 말고 각 항목의 정의와 계산 코드를 확인해야 합니다. 합성 원문 다섯 건은 "10", 빈 문자열, "20", "오류", "0"입니다. 수치는 세 건, 미측정은 한 건, 이슈는 한 건입니다. 입력 전체와 분류된 부분이 빠짐없이 맞는지 검사합니다.

이 프로그램의 관계는 $input=accepted+issues$, $accepted=numeric+missing$ 입니다. 다섯 건 사례에서는 $5=4+1$, $4=3+1$ 입니다. 평균은 숫자 세 건의 합 $10+20+0=30$ 을 숫자 건수 3으로 나누어 10입니다. 수락 건수 4로 나누면 7.5가 되어 미측정을 0처럼 처리합니다. 이슈 건수까지 포함해 5로 나누면 6이 됩니다. 같은 원문에서 분모만 바뀌도 업무 의미가 바뀌므로 평균 대상과 건수를 함께 보고합니다.

한 행의 추적은 원문→해석→분류→누적 순서입니다. 잘못된 글자가 나오면 이슈 목록에 원문을 남기고 continue로 다음 행으로 넘어갑니다. 문제 행이 수치 목록에도 들어가면 중복 분류가 되어 두 계산 관계가 깨질 수 있습니다. 모든 수치가 없을 때 평균은 None입니다. 0이라고 쓰면 실제 평균 0을 계산했다는 오해가 생깁니다. 헤더만 있는 입력은 전체 0건이고 모든 값이 빈 입력은 전체 기록은 있으나 수치가 0건입니다. 둘 다 평균이 없지만 같은 자료는 아닙니다. 결과 안내에는 입력 파일명·명령·실제 출력·미확인 범위·변경 이유를 남깁니다. 현재 검사는 합성 온도값 규칙을 확인하며 모든 필드나 센서의 실제 성능을 보증하지 않습니다.

혼자 풀어 볼 문제

복습·도전: 원문은 "5", "15", 빈 문자열, "뜨거움", 빈 문자열, "0"입니다. 전체·수치·미측정·이슈·수락 건수와 평균을 구하고 두 계산식을 쓰세요. 수치가 전혀 없는 경우의 평균 표현도 적으세요.

정답과 오답의 이유

전체 6, 수치 3, 미측정 2, 이슈 1, 수락 5입니다. $6=5+1$, $5=3+2$ 이고 평균은 $20 \div 3$ 을 소수 둘째 자리로 반올림한 6.67입니다. 수치가 없으면 None입니다. 수락 5를 분모로 쓰면 4가 되어 미측정을 포함한 잘못된 평균입니다.

다시 확인할 질문 세 가지

1. 수락 건수는 수치 건수인가요? 이 검사기에서는 수치와 미측정을 합하므로 결과의 각 항목이 무엇을 세는지 확인합니다.
2. continue가 필요한 이유는? 이슈로 분류한 행을 정상 처리로 보내지 않고 다음 행으로 넘어갑니다.
3. 평균 None과 0은 같은가요? None은 계산할 수치가 없음이며 0은 실제 수치 평균이 0이라는 값입니다.

개념. 검사기는 한 행씩 온도를 해석하여 정상 수치·미측정·이슈를 나눕니다. 함수는 한 값의 규칙을 담당하고 반복문은 여러 행에 같은 규칙을 적용합니다.

왜 필요한가. 함수·파일·예외를 하나의 업무에 연결합니다. 긴 코드는 준비·정의·행 처리·집계·저장으로 나눠 추적합니다. 전체를 암기하거나 빈 편집기에서 다시 작성하는 것이 목표는 아닙니다.

함께 보는 예. 실제 수치는 22.5·25.0·0.0으로 합 47.5, 개수 3, 평균 15.83입니다. 빈 온도 1건은 미측정, 뜨거움과 51.0은 원문 이슈로 남깁니다. 0도 실제 측정이므로 평균에 포함합니다.

헛갈리는 경우. 평균이 낮아 보인다는 이유로 0을 빼지 않습니다. 입력 원문과 정해 둔 처리 규칙을 확인합니다. 이 프로그램은 온도 규칙에 집중하며 모든 필드와 실제 측정의 진위를 검사하지 않습니다.

전체 코드

```
import csv
import json
from pathlib import Path

def parse_temperature(raw):
    if raw == "":
        return None
    value = float(raw)
    if value < 0 or value > 50:
        raise ValueError("범위 확인 필요")
    return value

base = Path(__file__).resolve().parent.parent
source = base / "data" / "bad_readings.csv"
with source.open(encoding="utf-8", newline="") as file:
    rows = list(csv.DictReader(file))
numbers = []
issues = []
missing = 0
for row in rows:
    try:
        value = parse_temperature(row["temperature_c"])
    except ValueError:
        issues.append({"sensor_id": row["sensor_id"], "raw": row["temperature_c"]})
        continue
    if value is None:
        missing = missing + 1
    else:
        numbers.append(value)
average = None
if len(numbers) > 0:
    average = round(sum(numbers) / len(numbers), 2)
summary = {"input": len(rows), "accepted": len(numbers) + missing,
           "missing": missing, "issues": len(issues), "average": average}
(base / "work" / "check_summary.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
(base / "work" / "check_issues.json").write_text(
```

```
    json.dumps(issues, ensure_ascii=False, indent=2), encoding="utf-8")
print(summary["input"], summary["accepted"], summary["missing"], summary["issues"])
print(summary["average"])
```

각 줄의 뜻

줄	코드	뜻
1	<code>import csv</code>	CSV 도구를 가져옵니다.
2	<code>import json</code>	JSON 도구를 가져옵니다.
3	<code>from pathlib import Path</code>	경로 도구를 가져옵니다.
4	<code>``</code>	정의 부분을 나누는 빈 줄입니다.
5	<code>def parse_temperature(raw):</code>	온도 해석 함수를 정의합니다.
6	<code>if raw == "":</code>	빈칸인지 확인합니다.
7	<code>return None</code>	미측정을 반환합니다.
8	<code>value = float(raw)</code>	숫자로 변환합니다.
9	<code>if value < 0 or value > 50:</code>	범위를 확인합니다.
10	<code>raise ValueError("범위 확인 필요")</code>	범위 문제를 알립니다.
11	<code>return value</code>	유효 숫자를 반환합니다.
12	<code>``</code>	정의와 실행을 나눕니다.
13	<code>base = Path(__file__).resolve().parent.parent</code>	course 주소를 정합니다.
14	<code>source = base / "data" / "bad_readings.csv"</code>	입력 파일을 선택합니다.
15	<code>with source.open(encoding="utf-8", newline="") as file:</code>	입력 파일을 엽니다.
16	<code>rows = list(csv.DictReader(file))</code>	헤더 키로 행을 읽습니다.
17	<code>numbers = []</code>	숫자 목록을 준비합니다.
18	<code>issues = []</code>	이슈 목록을 준비합니다.
19	<code>missing = 0</code>	미측정 개수를 준비합니다.
20	<code>for row in rows:</code>	한 행씩 처리합니다.
21	<code>try:</code>	해석을 시도합니다.
22	<code>value = parse_temperature(row["temperature_c"])</code>	이번 온도 원문을 함수에 줍니다.

줄	코드	뜻
23	<code>except ValueError:</code>	변환·범위 오류를 처리합니다.
24	<code>issues.append({"sensor_id": row["sensor_id"], "raw": row["temperature_c"]})</code>	센서명과 원문을 이슈로 보존합니다.
25	<code>continue</code>	다음 행으로 넘어갑니다.
26	<code>if value is None:</code>	미측정을 구별합니다.
27	<code>missing = missing + 1</code>	미측정 개수를 늘립니다.
28	<code>else:</code>	숫자인 경우입니다.
29	<code>numbers.append(value)</code>	수치 목록에 넣습니다.
30	<code>average = None</code>	수치가 없을 때의 평균을 준비합니다.
31	<code>if len(numbers) > 0:</code>	숫자가 있을 때만 계산합니다.
32	<code>average = round(sum(numbers) / len(numbers), 2)</code>	실제 수치 개수로 나누어 반올림합니다.
33	<code>summary = {"input": len(rows), "accepted": len(numbers) + missing,</code> <code>"missing": missing, "issues": len(issues), "average": average}</code>	입력·수락 건수로 요약의 시작합니다.
34	<code>summary = {"input": len(rows), "accepted": len(numbers) + missing,</code> <code>"missing": missing, "issues": len(issues), "average": average}</code>	미측정·이슈·평균을 포함합니다.
35	<code>(base / "work" / "check_summary.json").write_text(</code> <code>json.dumps(summary, ensure_ascii=False, indent=2),</code> <code>encoding="utf-8")</code>	요약 파일 쓰기를 시작합니다.
36	<code>(base / "work" / "check_issues.json").write_text(</code> <code>json.dumps(issues, ensure_ascii=False, indent=2),</code> <code>encoding="utf-8")</code>	이슈 파일 쓰기를 시작합니다.
37	<code>print(summary["input"], summary["accepted"],</code> <code>summary["missing"], summary["issues"])</code>	집계 네 개를 보여 줍니다.
38	<code>print(summary["average"])</code>	평균 또는 None을 보여 줍니다.
39		
40		

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A16_log_checker.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A16_log_checker.py
```

원본 코드의 예상 출력:

```
6 4 1 2
15.83
```

작은 수정: "bad_readings.csv"를 "readings.csv"로 바꿉니다.

수정 후 예상 출력:

```
4 4 1 0
15.83
```

학생 적용

1. 전체 코드를 역할별 다섯 구간으로 표시합니다.
2. 건수와 평균을 손으로 먼저 확인합니다.
3. 실행 뒤 결과 JSON 두 개를 엽니다.
4. 입력 파일명만 readings.csv로 바꿔 비교합니다.

예상 결과: 오류 포함 6/4/1/2, 정상 4/4/1/0. 평균은 모두 15.83.

확인 질문: continue의 역할은 무엇인가요?

막혔을 때: 한 행의 원문→함수→분류를 먼저 추적합니다. 여러 곳을 동시에 바꾸지 않습니다.

[활동 자료와 작성칸](#)에서 A16를 엽니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

A16 · 직접 해보기

A16 · 로그 검사기 전체 흐름

추가 원문·작업 파일:

- [A16_input.md](#)
- [A16_log_checker.py](#)
- [bad_readings.csv](#)
- [readings.csv](#)
- [A16_notes.md](#)
- [A16_log_checker.py](#)
- [check_summary.json](#)
- [check_issues.json](#)

목적: 함수·파일·예외를 하나의 업무에 연결합니다. 긴 코드는 준비·정의·행 처리·집계·저장으로 나눠 추적합니다. 전체를 암기하거나 빈 편집기에서 다시 작성하는 것이 목표는 아닙니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A16_input.md](#)
- [A16_log_checker.py](#)
- [A16_notes.md](#)
- [A16_log_checker.py](#)

단계

1. 전체 코드를 역할별 다섯 구간으로 표시합니다.
2. 건수와 평균을 손으로 먼저 확인합니다.
3. 실행 뒤 결과 JSON 두 개를 엽니다.
4. 입력 파일명만 readings.csv로 바꿔 비교합니다.

예상 결과: 오류 포함 6/4/1/2, 정상 4/4/1/0. 평균은 모두 15.83.

확인 방법: continue의 역할은 무엇인가요?

오류 해결: 한 행의 원문→함수→분류를 먼저 추적합니다. 여러 곳을 동시에 바꾸지 않습니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A17 · 정상·빈 입력·잘못된 입력 비교

개념. 검사기는 서로 다른 입력 집합으로 확인합니다. 헤더만 있는 파일, 전부 미측정인 파일, 숫자 오류가 있는 파일은 다른 경계 상황입니다.

왜 필요한가. 수치가 하나도 없으면 평균 분모가 0입니다. 평균 None은 계산할 수치가 없다는 결과이며 실제 평균 0.0과의 미가 다릅니다.

함께 보는 예. empty_readings.csv는 데이터 0건, missing_readings.csv는 미측정 2건입니다. 둘 다 평균 None이지만 입력수와 미측정수는 다릅니다.

헛갈리는 경우. None이면 모두 실패라고 판정하지 않습니다. 입력 조건에 따라 의도한 결과일 수 있습니다. 어떤 파일로 무엇을 확인했는지 기록합니다.

제공 입력

readings.csv4건 / bad_readings.csv6건 / empty_readings.csv헤더만 / missing_readings.csv미측정2건.
work/A16_log_checker.py 안 source 줄의 따옴표 속 CSV 이름만 바꿉니다. .py 이름은 유지합니다.

학생 적용

1. 네 입력의 기대 집계표를 씁니다.
2. work/A16_log_checker.py 안 source 줄의 따옴표 속 CSV 이름만 바꿔(.py 이름은 유지) 같은 프로그램을 실행합니다.
3. 평균None의 이유와 집계 관계를 확인합니다.
4. 헤더·다른 필드·실제 진위는 미검증으로 기록합니다.

예상 결과: empty 0/0/0/0·None, missing 2/2/2/0·None.

확인 질문: 수치가 없을 때 평균 0을 쓰면 어떤 오해가 생기나요?

막혔을 때: 이전 실행 결과 파일을 새 결과로 혼동하지 않도록 입력명·저장·실행 경로를 확인합니다.

[활동 자료와 작성칸](#)에서 A17를 엽니다.

검사할 CSV만 바꾸기

수정 파일은 `work/A16_log_checker.py`입니다. **.py 파일 이름은 그대로 둡니다.** 코드 안 `source = base / "data" / "bad_readings.csv"` 한 줄에서 따옴표 속 CSV 이름만 바꿉니다. 예: `source = base / "data" / "readings.csv"`.

저장 후 course 폴더에서 `python -X utf8 work/A16_log_checker.py`를 실행합니다. 실행마다 `work/check_summary.json`과 `work/check_issues.json`이 덮어써지므로 **결과를 먼저 기록한 뒤** 다음 CSV로 바꿉니다.

따옴표 속 입력 이름	예상 집계·평균	실제 집계·평균	직접 실행/제공 출력 관찰	차이와 근거
readings.csv	——	——	——	——
bad_readings.csv	——	——	——	——
empty_readings.csv	——	——	——	——
missing_readings.csv	——	——	——	——

평균을 낼 숫자가 없다는 결과와 실제 측정 0을 구분해 설명합니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: `readings.csv`의 정상 숫자 한 건과 `missing_readings.csv`의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 `A16_notes.md`, 네 입력 비교는 `A17_notes.md`에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 `work/A16_log_checker.py`를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 `solutions/A16_answer.md`와 `A17_answer.md`의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

A17 · 직접 해보기

A17 · 정상·빈 입력·잘못된 입력 비교

추가 원문·작업 파일:

- [A17_input.md](#)
- [readings.csv](#)
- [bad_readings.csv](#)
- [empty_readings.csv](#)
- [missing_readings.csv](#)
- [A17_notes.md](#)
- [A16_log_checker.py](#)

목적: 수치가 하나도 없으면 평균 분모가 0입니다. 평균 None은 계산할 수치가 없다는 결과이며 실제 평균 0.0과 의미가 다릅니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A17_input.md](#)

- [A17_notes.md](#)

단계

1. 네 입력의 기대 집계표를 씁니다.
2. work/A16_log_checker.py 안 source 줄의 따옴표 속 CSV 이름만 바꿔(.py 이름은 유지) 같은 프로그램을 실행합니다.
3. 평균None의 이유와 집계 관계를 확인합니다.
4. 헤더·다른 필드·실제 진위는 미검증으로 기록합니다.

예상 결과: empty 0/0/0/0·None, missing 2/2/2/0·None.

확인 방법: 수치가 없을 때 평균 0을 쓰면 어떤 오해가 생기나요?

오류 해결: 이전 실행 결과 파일을 새 결과로 혼동하지 않도록 입력명·저장·실행 경로를 확인합니다.

검사할 CSV만 바꾸기

수정 파일은 `work/A16_log_checker.py`입니다. **.py 파일 이름은 그대로 둡니다.** 코드 안 `source = base / "data" / "bad_readings.csv"` 한 줄에서 따옴표 속 CSV 이름만 바꿉니다. 예: `source = base / "data" / "readings.csv"`.

저장 후 course 폴더에서 `python -X utf8 work/A16_log_checker.py`를 실행합니다. 실행마다 `work/check_summary.json`과 `work/check_issues.json`이 덮어써지므로 **결과를 먼저 기록한 뒤** 다음 CSV로 바꿉니다.

따옴표 속 입력 이름	예상 집계·평균	실제 집계·평균	직접 실행/제공 출력 관찰	차이와 근거
readings.csv	——	——	——	——
bad_readings.csv	——	——	——	——
empty_readings.csv	——	——	——	——
missing_readings.csv	——	——	——	——

평균을 낼 숫자가 없다는 결과와 실제 측정 0을 구분해 설명합니다.

중간 확인과 다음 활동 합류

- 함께 확인할 증거: readings.csv의 정상 숫자 한 건과 missing_readings.csv의 미측정 한 건을 따라 읽고, 0은 숫자·미측정은 평균 대상이 아님을 설명합니다.
- 남은 작업: A16의 전체 구간 표시는 A16_notes.md, 네 입력 비교는 A17_notes.md에 미완료 항목을 표시하고 이어서 완성합니다. 이미 작성한 결과를 지우지 않습니다.
- 다음 입력: 제공된 work/A16_log_checker.py를 그대로 출발점으로 사용하고 A17에서 CSV 이름만 교체합니다. 실행이 막히면 solutions/A16_answer.md와 A17_answer.md의 제공 결과를 **관찰**로 기록하고 A18 인수인계에 미완료를 남깁니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 검사기 사용 방법과 검사 항목 전달하기

이 교사의 다른 예와 코드 해설 열기

오늘 배운 내용 확인

확인할 것	배점	충족 기준
배운 내용 설명	20점	사용한 개념과 선택한 이유를 예로 설명한다.
결과물 완성	40점	오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.
결과 확인	30점	예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.

저장과 전달	10점	파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다.
--------	-----	---

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: A16_notes.md, A16_log_checker.py, check_summary.json, check_issues.json / A17_notes.md, A16_log_checker.py

세 가지 확인 질문

1. print로 보여 주는 것과 return으로 돌려주는 것은 어떻게 다른가요?
내 설명: _____
2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.
내 설명: _____
3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.
내 설명: _____

더 해보기 · 추가 확인 문제

구간별 확인 질문

1. value는 모든 호출에서 같은 값을 뜻하나요?
2. 20과 30만 확인하면 놓칠 수 있는 실수는 무엇인가요?
3. print와 write_text는 같은 행동인가요?
4. except를 실행하면 실제 온도를 알아낸 것인가요?
5. 미측정도 분모에 넣어 4로 나누면 무엇이 바뀌나요?
6. 이슈 0건이라는 결과만 남기면 무엇이 부족한가요?

문제 1 · 함수가 받을 값과 돌려줄 결과 정하기

복습·도전: 물품 라벨에 상품명과 보관함을 받아 '장갑 / B02'처럼 한 문자열을 만드는 작업을 설계하세요. 입력 자리 두 개, 반환할 종류, 순서를 바꿨을 때 생길 문제를 문장으로 쓰세요. 첫 입력은 장갑/B02, 둘째 입력은 마스크/A01입니다. 이 두 호출의 결과를 예상하세요.

정답·해설: 입력은 상품명·보관함 문자열, 결과는 두 내용을 순서대로 연결한 문자열입니다. 장갑/B02와 마스크/A01의 결과는 각 입력에 따라 달라집니다. 순서를 바꾸면 B02 / 장갑처럼 업무 양식이 달라집니다. 컴퓨터가 이름의 뜻을 추측해 입력 위치를 고쳐 준다는 답은 오답입니다.

문제 2 · 화면 표시와 다음 계산의 반환값 구별하기

복습·도전: 배송 준비 함수는 포장비 500원을 화면에 표시하지만 반환문이 없습니다. 호출 결과를 fee에 연결한 뒤 총액 3000에 더하려 합니다. 화면값·fee 값·다음 계산을 분리해 설명하고 올바른 역할 배치를 문장으로 설계하세요.

정답·해설: 화면에는 500이 보이지만 fee는 None이며 다음 덧셈은 TypeError가 날 수 있습니다. 함수가 500을 반환하고 호출자가 총액에 더한 뒤 표시하도록 설계합니다. 화면에 500이 보이므로 fee도 500이라는 답은 표시와 반환을 혼동한 것입니다.

문제 3 · 읽은 텍스트에서 값으로, 값에서 저장 파일로

복습·도전: 입력 헤더는 device_id,value이고 한 행은 D09,0입니다. 프로그램은 temperature_c를 찾다가 실패했습니다. 파일 열기·헤더 찾기·숫자 변환 중 어느 단계인지 판단하고 원문을 보존한 수정 계획과 올바른 JSON 수치 표현을 쓰세요.

정답·해설: 헤더 찾기 단계의 문제입니다. 입력 열 이름이 value로 정해져 있다면 코드에서 찾는 키도 value로 맞춥니다. 고치기 전에 정해 둔 열 이름과 실제 헤더를 확인합니다. 수치 0은 실제 값이므로 JSON에서는 0이며 null이 아닙니다. 주소 수정이나 0 삭제는 현재 증거에 맞는 해결이 아닙니다.

문제 4 · 미측정·잘못된 글자·실제 0의 세 갈래

복습·도전: 수량 칸 원문이 빈 문자열, "0", "여섯", " "입니다. 이 문제의 규칙은 정확한 빈 문자열만 미확인으로 보고, 숫자만 계산 대상으로 정합니다. 네 갈래의 분류와 담당자에게 남길 이슈 원문을 작성하세요.

정답·해설: 빈 문자열은 미확인, "0"은 수치 0, "여섯"과 공백 한 칸은 현재 규칙으로 숫자로 읽을 수 없는 입력입니다. 두 이슈 원문을 그대로 남깁니다. 공백을 자동 삭제해 미확인으로 합치는 답은 명시되지 않은 정리 규칙을 추가한 것입니다.

문제 5 · 허용 범위의 바깥과 경계를 검사하기

복습·도전: 수업용 배터리 기록의 허용 충전율은 0~100%입니다. -0.1, 0, 50, 100, 100.1의 기대 결과와 바깥을 찾는 논리 문장을 쓰세요. 100.1을 100으로 자동 변경해도 되는지 근거를 설명하세요.

정답·해설: 양 끝 -0.1·100.1은 거부, 0·50·100은 허용입니다. '0 미만 또는 100 초과'가 바깥입니다. 자동 변경은 원문을 바꾸므로 현재 검사 규칙에서는 허용하지 않습니다. and로 쓰면 두 바깥 조건을 동시에 만족하는 값이 없어 오류를 놓칩니다.

문제 6 · 분류 건수와 평균 분모를 보존하기

복습·도전: 원문은 "5", "15", 빈 문자열, "뜨거움", 빈 문자열, "0"입니다. 전체·수치·미측정·이슈·수락 건수와 평균을 구하고 두 검산식을 쓰세요. 수치가 전혀 없는 경우의 평균 표현도 적으세요.

정답·해설: 전체 6, 수치 3, 미측정 2, 이슈 1, 수락 5입니다. 6=5+1, 5=3+2이고 평균은 20÷3을 소수 둘째 자리로 반올림한 6.67입니다. 수치가 없으면 None입니다. 수락 5를 분모로 쓰면 4가 되어 미측정을 포함한 잘못된 평균입니다.